PROJECT REPORT · 2026.11

Remote-Desktop-TS

Python · TypeScript · FastAPI · PostgreSQL · tkinter로 만든
웹 기반 원격 데스크톱 — 프로젝트 결과 보고서

Python | FastAPI | TypeScript | zod | PostgreSQL | WebSocket | tkinter | PyInstaller | GitHub Actions

~30 fps
실시간 화면 전송 (WebSocket)

3계층
동일 스키마로 송수신 타입 고정

.exe
Python 설치 없이 실행하는 호스트 앱

4 / 4
통합 테스트 통과 (실제 PostgreSQL)

저장소 github.com/Ryu-Taehyun/Remote-Desktop-TS

작성 Ryu-Taehyun · 공동 작업 Claude

01 프로젝트 개요

한 컴퓨터의 화면을 다른 컴퓨터의 브라우저에서 실시간으로 보고, 허용된 경우 마우스와 키보드로 원격 조작하는 프로그램입니다.

화면을 공유하는 쪽은 Python GUI 앱(Windows용 `.exe` 제공)을 실행하고, 보는 쪽은 별도 설치 없이 브라우저로 접속합니다. 두 쪽은 FastAPI 서버를 통해 연결됩니다. 서버는 회원가입·로그인, 세션과 접속 코드 관리, 화면·입력 중계를 맡고, 사용자 데이터는 PostgreSQL에 저장합니다.

27–30 fps
실시간 모드 화면 속도
(1280px, JPEG 품질 60)

약 7배
최초 구현(약 4 fps)
대비 속도 향상

6종
허용된 입력 이벤트 타입
(그 외는 서버가 거부)

1개 포트
API·WebSocket·웹 화면을
서버 하나(:8000)로 제공

요구사항과 구현 결과

요구사항	구현 결과
Python · FastAPI	컴퓨터 간 데이터(화면 프레임, 입력 이벤트) 수신·중계, 사용자·세션 데이터 DB 저장, JWT 인증
TypeScript	로그인·회원가입, 세션 목록, 원격 화면 뷰어. zod 스키마로 **미리 지정된 타입만 송수신**
PostgreSQL	사용자(bcrypt 해시 비밀번호), 원격 세션, 세션 참여자 저장
Python GUI	tkinter 호스트 앱: 로그인, 세션 생성·접속 코드, 화면 공유, 원격 입력 재생. PyInstaller로 단일 `.exe` 배포
GitHub	공개 저장소에 커밋·푸시, GitHub Actions로 `.exe` 자동 빌드 및 Releases 배포, 프로필 README에 프로젝트 등록

기술 스택

영역	사용 기술
백엔드	FastAPI, Pydantic v2, SQLAlchemy 2.0, psycopg 3, uvicorn(WebSocket), PyJWT, bcrypt
프론트엔드	TypeScript 5.8 (strict), Vite 6, zod 3
호스트 앱	tkinter, mss(화면 캡처), Pillow(JPEG), pynput(입력), websockets(동기 클라이언트), requests
데이터베이스	PostgreSQL 16 이상
배포	PyInstaller(단일 .exe), GitHub Actions, GitHub Releases

02 시스템 구성

모든 데이터는 서버를 거칩니다. 호스트 앱과 브라우저는 각각 서버에 WebSocket으로 연결하고, 서버는 화면 프레임을 뷰어에게, 입력 이벤트를 호스트에게 즉시 전달합니다.

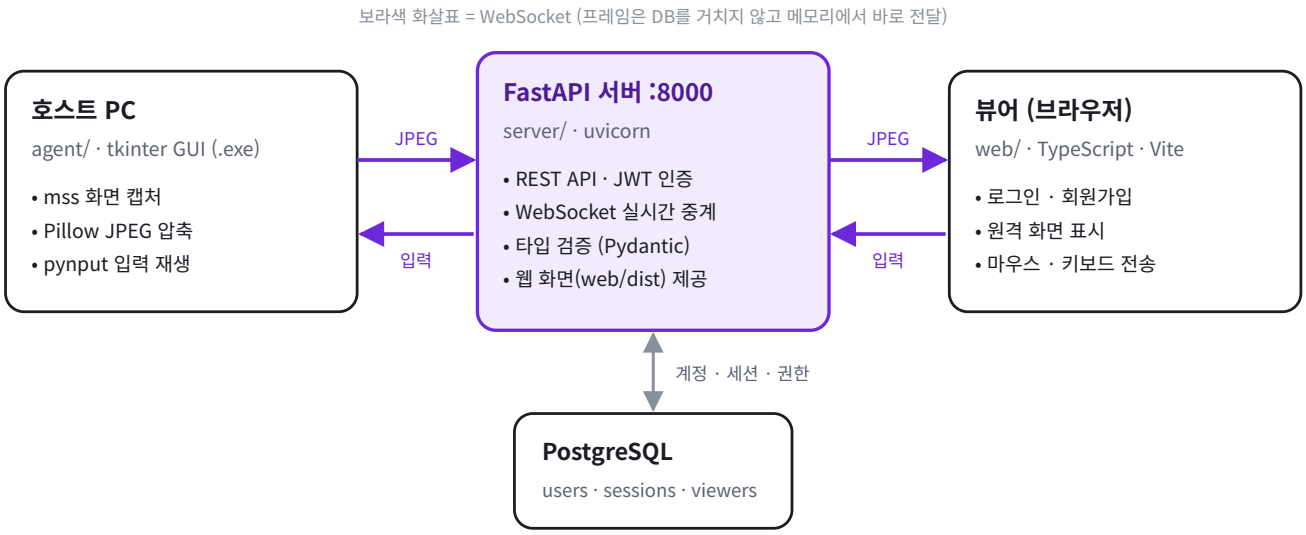


그림 1. 전체 구성. 호스트 PC와 뷰어는 서로 직접 연결되지 않고 서버를 통해 통신합니다.

구성 요소

폴더	기술	역할
`server/`	Python · FastAPI · SQLAlchemy	REST API, WebSocket 중계(`realtime.py`), 인증, DB 접근, 빌드된 웹 화면 제공
`web/`	TypeScript · Vite · zod	로그인·회원가입, 세션 목록, 원격 화면 뷰어 (프레임워크 없이 순수 TypeScript)
`agent/`	Python · tkinter · mss · pynput	호스트 앱: 화면 캡처·압축·전송, 원격 입력 재생, 서버 주소·이메일 기억
`tests/`	pytest · TestClient	실제 PostgreSQL을 사용하는 HTTP·WebSocket 통합 테스트
`.github/`	GitHub Actions	Windows에서 PyInstaller로 `RemoteDesktopHost.exe` 빌드 후 Releases에 게시

03 주요 기능

AUTH

회원가입 · 로그인

이메일·이름·비밀번호로 가입하며 비밀번호는 bcrypt로 해시해 저장합니다. 로그인하면 JWT(유효 12시간)를 발급하고 이후 모든 요청에 Bearer 토큰으로 사용합니다. 웹과 호스트 앱 모두 같은 계정을 씁니다.

SESSION

세션 · 접속 코드

호스트가 공유를 시작하면 8자리 접속 코드(혼동되는 0/O, 1/I 제외)가 발급됩니다. 뷰어는 로그인 후 코드를 입력해 참여하고, 참여한 세션은 목록에 남아 다시 볼 수 있습니다. 세션을 종료하면 코드가 무효화됩니다.

STREAM

실시간 화면 공유

호스트 앱이 화면을 캡처·축소·JPEG 압축해 WebSocket으로 보내고, 서버가 메모리에서 바로 뷰어에게 전달합니다. FPS(1–30), JPEG 품질, 최대 너비를 앱에서 조절할 수 있습니다.

CONTROL

원격 제어

호스트가 **원격 제어 허용**을 켜야만 입력이 전달됩니다. 뷰어 화면을 클릭하면 마우스 이동·클릭·휠, 키보드 입력이 0–1로 정규화된 좌표와 함께 호스트로 전송되어 실제 입력으로 재생됩니다.

HOST APP

호스트 앱 (.exe)

Python 설치 없이 더블클릭으로 실행됩니다. 마지막 서버 주소와 이메일을 기억해 다음 실행 때 자동으로 채웁니다(비밀번호는 저장하지 않음). 전송 방식·fps·프레임 크기를 상태줄에 표시합니다.

FALLBACK

자동 호환 모드

프록시·방화벽으로 WebSocket이 막힌 환경에서는 호스트 앱과 뷰어가 자동으로 HTTP 폴링 방식으로 전환합니다. 두 방식의 클라이언트가 섞여 있어도 서로 화면과 입력을 주고받습니다.

화면

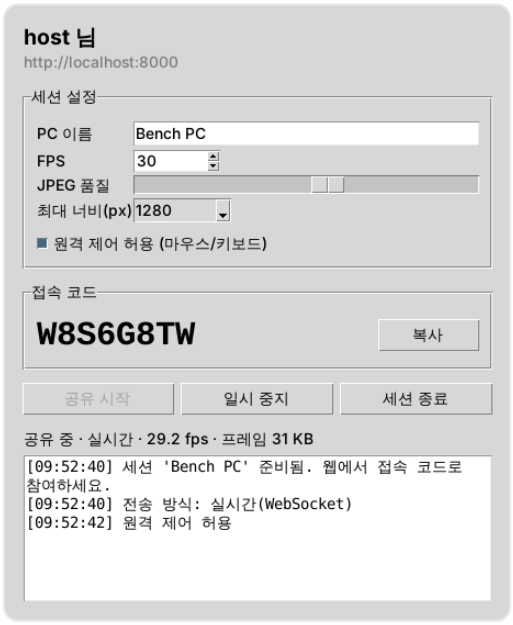


그림 2. 호스트 앱 — 접속 코드, 전송 설정, 실시간 상태 (29 fps)

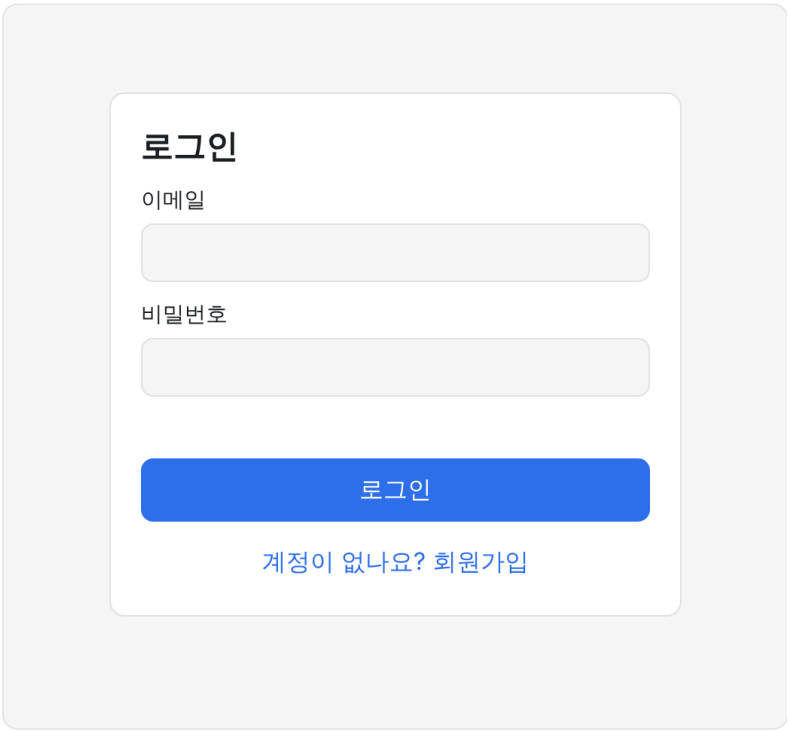


그림 3. 웹 로그인 · 회원가입 화면

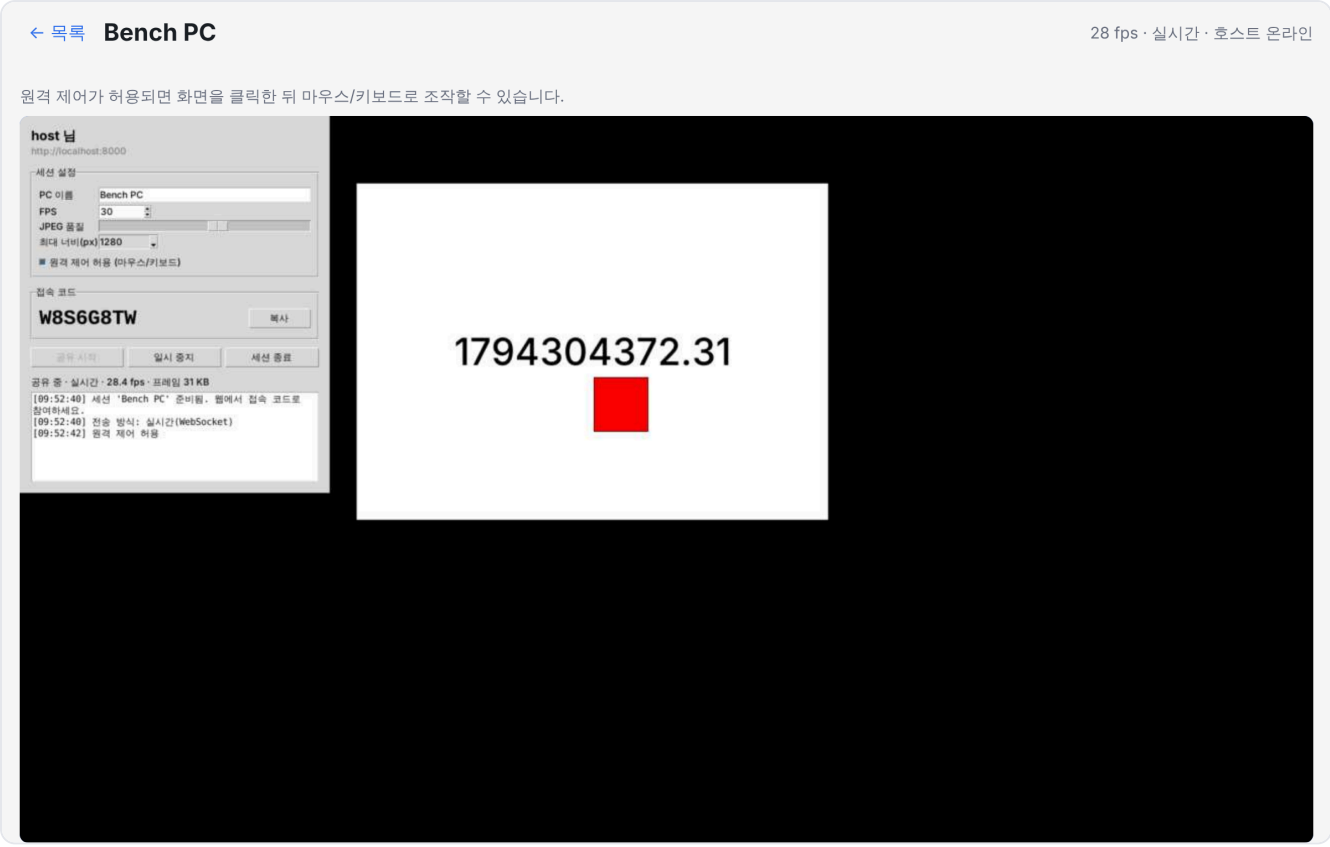


그림 4. 원격 화면 뷰어 — 호스트 PC의 화면을 실시간(28 fps)으로 표시. 오른쪽 위에 fps, 전송 방식, 호스트 상태가 표시됩니다.

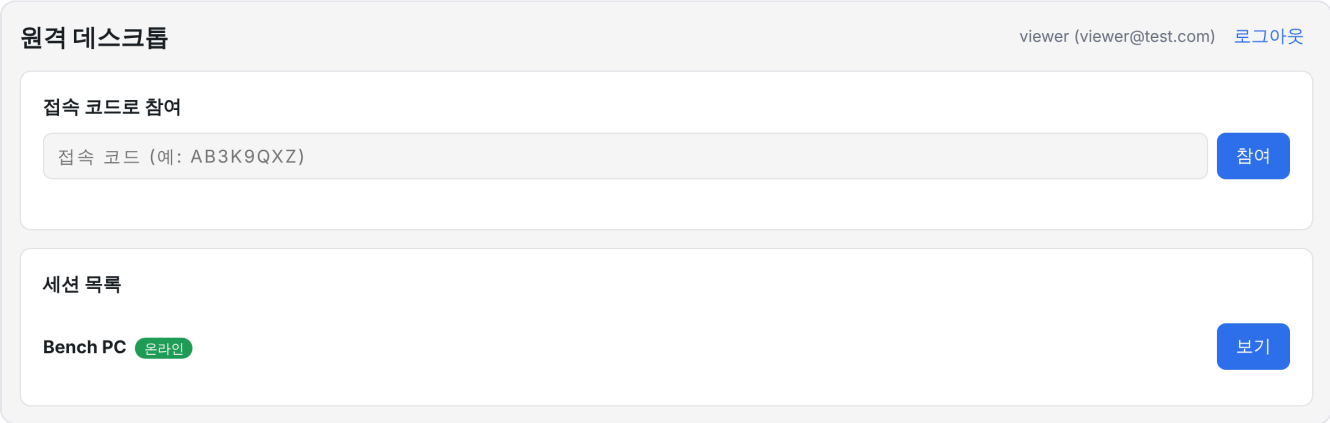


그림 5. 세션 목록 — 접속 코드로 참여하거나, 이미 참여한 세션을 다시 엽니다.

04 타입 고정 송수신

요구사항인 “미리 지정된 타입만 송수신”을 세 계층 모두에서 같은 스키마로 구현했습니다. 정의되지 않은 필드나 메시지는 보내지도, 받지도 않습니다.

파일	라이브러리	역할
`server/schemas.py`	Pydantic `extra="forbid"`	모든 요청·WebSocket 메시지 검증. 위반 시 422 또는 `error` 메시지로 거부
`web/src/protocol.ts`	zod `.strict()`	요청은 **보내기 전에**, 응답은 **받은 뒤에** 검증. TypeScript 타입도 스키마에서 생성
`agent/protocol.py`	Pydantic	호스트 앱이 받는 입력 이벤트와 서버 메시지 검증

허용되는 메시지

구분	허용 타입
입력 이벤트	`mouse_move` `mouse_down` `mouse_up` `wheel` `key_down` `key_up` 좌표는 0–1 범위, 버튼은 left/right/middle, 휠은 ±50, 키 이름은 32자 이내로 제한
WebSocket 텍스트	`auth` `frame_meta` `input` `status` `error`
WebSocket 바이너리	JPEG 프레임만 허용 (파일 시그니처 `FF D8` 확인, 최대 3MB)

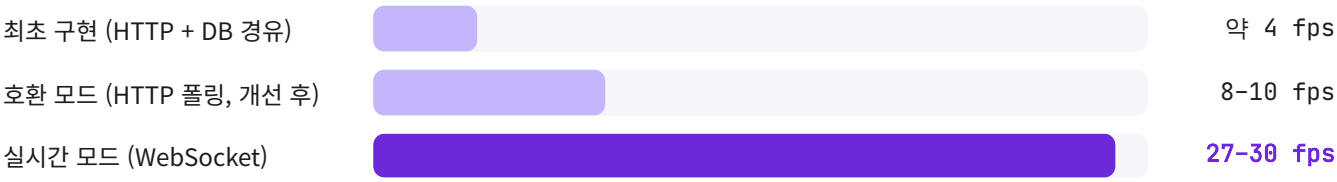
> 테스트에서 `{"type": "shell_exec", "cmd": "…"}` 같은 정의되지 않은 메시지, 범위를 벗어난 좌표, JPEG이 아닌 바이너리가 모두 거부되는 것을 확인했습니다.

API 목록

메서드	경로	설명
POST	/api/auth/signup · /login	회원가입 · 로그인 (JWT 발급)
GET	/api/auth/me	내 정보
POST · GET	/api/sessions	세션 생성 · 내 세션 목록
POST	/api/sessions/join	접속 코드로 참여
PATCH · DELETE	/api/sessions/{id}	원격 제어 허용 변경 · 세션 종료 (소유자 전용)
WS	/api/ws/host/{id}	호스트 실시간 연결: 프레임 송신, 입력 수신
WS	/api/ws/view/{id}	뷰어 실시간 연결: 프레임 수신, 입력 송신
PUT · GET	/api/sessions/{id}/frame	호환 모드 프레임 업로드 · 조회
POST · GET	/api/sessions/{id}/input	호환 모드 입력 전송 · 가져오기
GET	/api/health · /api/health/db	서버 · DB 상태 확인

05 실시간 전송과 성능

같은 조건(1920×1080 화면을 1280px로 축소, JPEG 품질 60, 계속 움직이는 화면)에서 뷰어가 실제로 받은 초당 프레임 수를 측정했습니다.



프레임 크기는 약 31–33KB. 실시간 모드는 호스트 앱 FPS 설정 30, 호환 모드는 10에서 측정. 두 모드 모두 원격 클릭이 정확한 위치(1920×1080 기준 (10%, 90%) → (191, 970))에 재생됨을 확인.

속도를 높인 설계

항목	내용
메모리 중계	실시간 모드에서는 프레임이 DB를 거치지 않고 서버 메모리에서 바로 뷰어에게 전달됩니다.
최신 프레임 우선	뷰어가 느리면 서버와 브라우저가 중간 프레임을 버리고 항상 가장 최신 화면만 표시해 지연이 쌓이지 않습니다.
변화 없는 화면 생략	화면이 그대로면 프레임을 보내지 않고, 연결 유지를 위해 1초에 한 번만 전송합니다.
즉시 입력	클릭·키 입력은 바로, 마우스 이동은 마지막 위치만 30ms 간격으로 묶어 보냅니다.
빠른 인코딩	단계적 축소(`reducing_gap`)와 최적화 옵션 제거로 캡처·압축 시간을 줄였습니다. JPEG은 이미 압축돼 있으므로 WebSocket 압축은 끕니다.
자동 전환	WebSocket 연결에 실패하면 HTTP 폴링으로 전환하고, 연결이 끊기면 자동으로 다시 연결합니다.

데이터베이스

테이블	내용
`users`	id(UUID), email(고유), username, password_hash(bcrypt), created_at
`remote_sessions`	id, owner_id, name, access_code(고유), allow_control, active, last_seen
`session_viewers`	세션에 참여한 사용자 (session_id + user_id 고유)
`frames · input_events`	호환 모드용 최신 프레임과 입력 대기열 (실시간 모드에서는 사용하지 않음)

테이블은 서버 첫 요청 때 자동 생성되며, DDL은 `db/schema.sql`에도 정리되어 있습니다.

06 보안

항목	적용 내용
비밀번호	bcrypt 해시로 저장, 원문은 저장하지 않음. 호스트 앱도 비밀번호를 기억하지 않음
인증	JWT(HS256). 서명 키는 `JWT_SECRET` 또는 최초 실행 시 자동 생성되는 `.jwt_secret`(git 제외)
WebSocket 인증	토큰을 URL에 넣지 않고 연결 직후 첫 메시지(`auth`)로 전송해 로그에 남지 않게 함
화면 접근 권한	세션 소유자와 접속 코드로 참여한 로그인 사용자만 화면을 볼 수 있음
원격 제어 권한	호스트가 허용했을 때만 입력이 전달·재생됨. 허용을 끄면 대기 중인 입력도 삭제. 허용 시 확인 창 표시
소유자 전용 동작	프레임 업로드, 입력 가져오기, 허용 변경, 세션 종료는 소유자만 가능
입력 검증	모든 메시지를 스키마로 검증하고, 정의되지 않은 타입·필드·범위 밖 값은 거부
XSS 방지	웹 화면은 사용자 데이터를 `textContent`로만 넣고 `innerHTML`을 쓰지 않음
개인정보	공개 저장소 커밋에는 실제 이메일 대신 GitHub noreply 주소 사용

07 배포와 실행

GitHub

항목	결과
저장소	`github.com/Ryu-Taehyun/Remote-Desktop-TS` (Public, `main`)
자동 빌드	`main`에 호스트 앱 코드가 push되면 GitHub Actions가 Windows에서 `RemoteDesktopHost.exe`(약 24MB)를 빌드
릴리스	빌드 결과를 Releases에 자동 게시 (현재 최신 **v1.0.3**)
프로필	GitHub 프로필 README의 Systems & Algorithms 표에 프로젝트 등록

실행 방법 (Windows, 같은 네트워크)

```
# 1. 최초 1회: 패키지 설치와 웹 빌드
python -m venv .venv ; .venv\Scripts\Activate.ps1
python -m pip install -r requirements-dev.txt -r agent/requirements.txt
cd web ; npm install ; npm run build ; cd ..

# 2. 서버 실행 (API · WebSocket · 웹 화면을 :8000 하나로 제공)
python -m uvicorn server.main:app --host 0.0.0.0 --port 8000
```

- **공유할 PC**: `RemoteDesktopHost.exe` 실행 → 서버 주소 `http://<서버 IP>:8000` 입력 → 로그인 → **공유 시작**

- **보는 PC**: 브라우저로 `http://<서버 IP>:8000` 접속 → 로그인 → 접속 코드 입력
- 원격 조작은 호스트 앱에서 **원격 제어 허용**을 켠 뒤 뷰어 화면을 클릭해서 시작

검증

항목	결과
통합 테스트 (pytest, 실제 PostgreSQL)	**4개 통과** — 인증, 타입 거부, HTTP 세션 전체 흐름, WebSocket 실시간 흐름(권한·타입 거부·HTTP 혼용·세션 종료 알림)
TypeScript	**strict 타입 검사 · 빌드 통과**
종단 간 테스트	**통과** — 가상 디스플레이에서 호스트 앱 → 서버 → 브라우저까지 화면 수신과 원격 클릭 위치 확인
.exe 빌드	**GitHub Actions 전 단계 성공**, Releases에 첨부 확인

08 한계와 향후 과제

현재 한계	개선 방향
서버가 켜져 있어야 함	모든 통신이 서버를 거치므로 서버 PC가 꺼지면 사용 불가 → 서버 자동 실행 스크립트, 상시 가동 PC·미니 PC에 설치
같은 네트워크 전제	HTTP(암호화 없음)로 동작 → 외부 공개 시 HTTPS 리버스 프록시(Caddy, nginx) 적용
로그인 시도 제한 없음	비밀번호 대입 공격 방지를 위한 rate limit 추가
.exe 코드 서명 없음	첫 실행 시 SmartScreen 경고 → 코드 서명 인증서 적용
단일 서버 프로세스	실시간 상태가 메모리에 있어 워커 1개로 실행 → 대규모 사용 시 Redis 등 외부 브로커로 분리
JPEG 프레임 방식	정지 화면 중심에는 효율적이나 영상에는 대역폭이 큼 → 변경 영역만 전송하거나 H.264 등 영상 코덱 적용